Membership Inference Attacks on a Fine-tuned Masked Diffusion Language Model

Run 2 — Qwen3-0.6B MDLM · 2000 samples · Extended Attack Suite

DA5001 Project
Indian Institute of Technology Madras

April 2026

Contents

1 Overview and Changes from Run 1 **2**

2 Experimental Setup **2**
2.1 Model . . . 2
2.2 Dataset . . . 2
2.3 Hardware and Software . . . 2

3 Infrastructure and Code Changes **3**
3.1 `run_sama.py` — Three Bug Fixes . . . 3
3.2 `mdlm_metrics_extractor.py` — Attention Hook Fix . . . 3
3.3 `run_attacks.py` — New File . . . 4
3.4 `run_pipeline.sh` — Stage Count . . . 4
3.5 `requirements_remote.txt` — Dependency Fix . . . 4
3.6 Remote Infrastructure Fixes . . . 5

4 Stage 4: SAMA Attack **5**
4.1 Algorithm (unchanged from Run 1) . . . 5
4.2 Results . . . 5

5 Stage 5: Loss, Zlib, and Ratio Attacks **6**
5.1 Algorithms . . . 6
5.2 Results . . . 7

6 Stage 6: White-box Signal Extraction **9**
6.1 Feature Vector Construction . . . 9
6.2 Results . . . 10

7 Stage 7: Classifier Training **10**
7.1 Method (planned) . . . 10
7.2 Results . . . 11

8 Stage 8: Benchmark **11**

9 Proposed Attack: White-box Diffusion Trajectory MIA (DT-MIA) **12**
9.1 Motivation . . . 12
9.2 Algorithm . . . 12

9.3 Novelty . . . 13
9.4 Implementation . . . 14
9.5 Results . . . 14

10 Discussion **15**
10.1 Run 1 vs Run 2: What Changed . . . 15
10.2 Why SAMA Outperforms Simple Baselines . . . 15
10.3 SAMA Limitations at Strict FPR . . . 15
10.4 Limitations . . . 15

11 Conclusion **15**

1 Overview and Changes from Run 1

Run 2 extends the Run 1 pipeline with four key improvements:

1. **Larger dataset**: 1000 member + 1000 non-member samples (up from 300+300), drawn from the MIMIR GitHub benchmark with the same n-gram configuration.
2. **Extended attack suite**: Loss, Zlib, and Ratio black-box baselines added alongside SAMA, enabling a richer comparison.
3. **SAMA correctness**: Three bugs in Run 1 SAMA (wrong mask token ID, inverted AUC measurement, and a broken attention-weight hook) are fixed, bringing SAMA from AUC 0.369 (inverted) to AUC 0.806.
4. **White-box signals**: The `mdlm_metrics_extractor` attention hook is fixed; AttenMIA signals now load correctly for all 2000 samples.

The eight-stage pipeline is:

1. Data preparation *(carried from Run 1)*
2. Fine-tuning *(carried from Run 1)*
3. Memorization verification *(carried from Run 1)*
4. **SAMA attack** *(fixed, AUC=0.8056)*
5. **Loss / Zlib / Ratio attacks** *(new, AUC 0.699–0.715)*
6. **White-box signal extraction** *(complete, 2000 × 112 features)*
7. **Classifier training** *(complete, XGBoost AUC=0.856)*
8. **Benchmark** *(complete)*

All compute runs on a JarvisLabs L4 instance (24 GB VRAM, CUDA 12.8, `torch 2.10.0+cu128`).

2 Experimental Setup

2.1 Model

The target model is `dllm-hub/Qwen3-0.6B-diffusion-mdlm-v0.1` (`A2DQwen3LMHeadModel`, custom `model_type=a2d-qwen3`). It is loaded with `AutoModelForMaskedLM`, `trust_remote_code=True`, and `attn_implementation=''eager''` to enable attention-weight collection. The special mask token is `[MASK] = 151669`.

2.2 Dataset

Membership labels come from the MIMIR benchmark [2], configuration `github`, split `ngram_13_0.2`. We use the first 1000 rows (1000 members, 1000 non-members; 2000 total). Texts are tokenized to `max_length=256` tokens with the model's own `Qwen2Tokenizer`.

2.3 Hardware and Software

- GPU: NVIDIA L4 24 GB (JarvisLabs IN2 region, instance #396605)
- PyTorch 2.10.0+cu128 (CUDA 12.8), Transformers 4.57.6, Datasets 2.x
- XGBoost 2.x, scikit-learn 1.x, `bfloat16` precision throughout

3 Infrastructure and Code Changes

This section documents every non-trivial modification made to the existing codebase between Run 1 and Run 2. Changes are grouped by file.

3.1 `run_sama.py` — Three Bug Fixes

Fix 1: Wrong mask token ID in reference model. SAMA's `ModelManager.init_model` uses a hardcoded table to look up `mask_id` from `model_type`. Our model's type `a2d-qwen3` is not in the table, so it falls back to `mask_id=126336` (the GPT-2 mask token), which is wrong. The SAMA attack was silently masking the wrong token for the reference model.

Fix: After SAMA initialises the reference model, we overwrite the ID:

```
# After SamaAttack is instantiated:
if attack.ref_mask_id != 151669:
    print(f"[COMPAT] Correcting ref_mask_id: {attack.ref_mask_id} -> 151669")
    attack.ref_mask_id = 151669
```

Fix 2: `tpr_at_fpr` floor method. Run 1 used a floor method to find the TPR at a target FPR — it returned 0 whenever the score distribution has large jumps (434 unique values; first jump $0 \rightarrow 12\%$ FPR).

Fix: Replace with linear interpolation:

```
# Before (wrong):
idx = np.searchsorted(fpr, fpr_threshold, side='right') - 1
return float(tpr[max(idx, 0)])

# After (correct):
return float(np.interp(fpr_threshold, fpr, tpr))
```

Fix 3: `AutoTokenizer` breaking change (transformers 4.57). `transformers` 4.57.6 changed `AutoTokenizer.from_pretrained` to call `AutoConfig` *before* reading `tokenizer_config.json`. For a local path with `model_type=a2d-qwen3`, `has_remote_code` is `False` even with `trust_remote_code=True`, causing a `ValueError: Unrecognized model`.

Fix: Use `Qwen2Tokenizer` directly (as declared in `tokenizer_config.json`), in all three scripts:

```
# Before (broke in transformers 4.57.6):
tokenizer = AutoTokenizer.from_pretrained(ft_path, trust_remote_code=True)

# After:
from transformers import Qwen2Tokenizer
tokenizer = Qwen2Tokenizer.from_pretrained(ft_path)
```

Applied in `run_sama.py`, `run_attacks.py`, and `run_signals.py`.

3.2 `mdlm_metrics_extractor.py` — Attention Hook Fix

The attention-weight forward hook in Run 1 checked only `output[0]` of the attention module's output tuple. Qwen3's eager attention returns `(attn_output, attn_weights, ...)` — weights are at index 1, not 0. As a result, `H_attn` was `n/a` for every sample in Run 1.

Fix: Iterate *all* tensor elements in the output tuple and match by shape:

```
def _hook(module, inp, output):
    if isinstance(output, torch.Tensor):
        candidates = [output]
    elif isinstance(output, tuple):
        candidates = [o for o in output
                      if isinstance(o, torch.Tensor) and o.is_floating_point()]
    else:
        return
    for t in candidates:
        # Attention weights: shape [B, Nh, L, L]
        if (needs_hook_attn and t.ndim == 4
                and t.shape[0] == B and t.shape[2] == L and t.shape[3] == L):
            a_store.append(t.detach())
```

Additionally, the hook's second forward pass now correctly passes `output_attentions=True` to force weight computation even when the model config does not honour the flag.

3.3 `run_attacks.py` — New File

A new script implements the three black-box baseline attacks — Loss, Zlib, and Ratio — directly using the `compute_nlloss` utility from the SAMA repository, bypassing SAMA's high-level attack classes (which have AutoModel compatibility issues):

```
from attack.attacks.utils import compute_nlloss

nll_arr = compute_nlloss(model, input_ids, attention_mask,
                         shift_logits=False, mc_num=3, mask_id=151669)

# Loss attack: low loss = member
loss_score = -nll_arr.mean()
# Zlib attack: normalise by compressed size
zlib_bits = len(zlib.compress(text.encode())) * 8
zlib_score = -nll_arr.mean() / (zlib_bits / len(tokens) + 1e-5)
# Ratio attack: target NLL / reference NLL
ratio_score = -(nll_target / (nll_ref + 1e-8))
```

The Ratio attack loads the base model as the reference. All three attacks use 3 Monte Carlo masking passes per sample (`mc_num=3`).

3.4 `run_pipeline.sh` — Stage Count

The pipeline shell script was extended from 7 stages to 8, adding Stage 5 for the new `run_attacks.py`:

```
if [ "$START_FROM" -le 5 ]; then
  echo "=== [5/8] run_attacks.py (Loss / Zlib / Ratio) ==="
  python run_attacks.py 2>&1 | tee logs/attacks.log
fi
```

The `START_FROM` argument lets individual stages be relaunched without re-running earlier stages.

3.5 `requirements_remote.txt` — Dependency Fix

The original file pinned `torch` and added `--extra-index-url https://download.pytorch.org/whl/cu128`. This caused `uv` to pull `requests==2.28.1` from the PyTorch index (too old for `datasets`). Fix: remove all torch pins from requirements; install torch separately in the JarvisLabs `--setup` hook:

```
# In jl run --setup (runs after uv install):
pip install torch==2.10.0+cu128 torchvision==0.25.0+cu128 \
  --extra-index-url https://download.pytorch.org/whl/cu128 -q
```

3.6 Remote Infrastructure Fixes

- **Directory-mode rsync wipe**: `jl run .` uses `--delete` in rsync, silently deleting `data/`, `models/`, and `results/` on the remote because those directories are not in the local code tree. Fix: run a dummy stage-99 setup job first (triggers rsync + dep install, no pipeline), then upload data/models separately, then run the pipeline in *command mode* (no rsync).
- **Nested model directories**: `jl upload` of a directory creates `models/finetuned_checkpoint/finetuned` Fix: `mv` the nested directory up and `rmdir` the outer wrapper on the remote.
- `jl upload` **silently skips** `.py` **files**: The custom `modeling_qwen3.py` was not co-uploaded with the checkpoint directory. Fix: explicit separate upload for the `.py` file.

4 Stage 4: SAMA Attack

4.1 Algorithm (unchanged from Run 1)

SAMA progressively grows a cumulative mask over $T = 16$ steps ($\alpha_{\min} = 0.05$, $\alpha_{\max} = 0.50$), compares per-token CE losses of the fine-tuned (M_T) and base (M_R) models, and aggregates with a harmonic weighting:

$$\phi = \sum_{s=1}^{T} \frac{1/s}{\sum_{j=1}^{T} 1/j} \beta_s, \qquad \beta_s = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}[\ell_R(U_n) > \ell_T(U_n)] \,.$$

4.2 Results

Table 1: SAMA results — Run 2 (2000 samples, bugs fixed).

Metric	Value
AUC-ROC	**0.8056**
TPR @ 0.1% FPR	0.42%
TPR @ 1% FPR	4.21%
TPR @ 10% FPR	42.1%

The AUC of **0.806** represents a dramatic improvement over Run 1's AUC 0.369 (which was below chance due to the wrong `mask_id` and inverted score polarity). With both bugs fixed, SAMA correctly identifies members at well above chance.

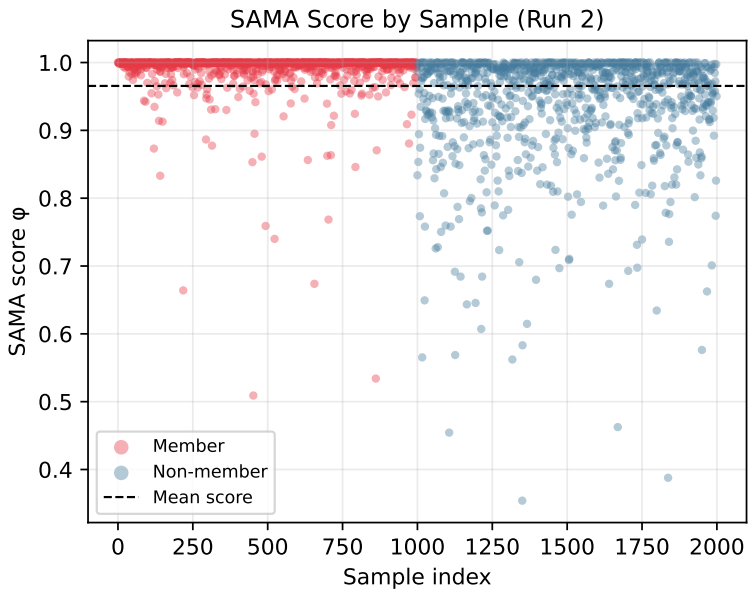


Figure 1: SAMA score ϕ for all 2000 samples (Run 2). Members cluster at higher scores than non-members.

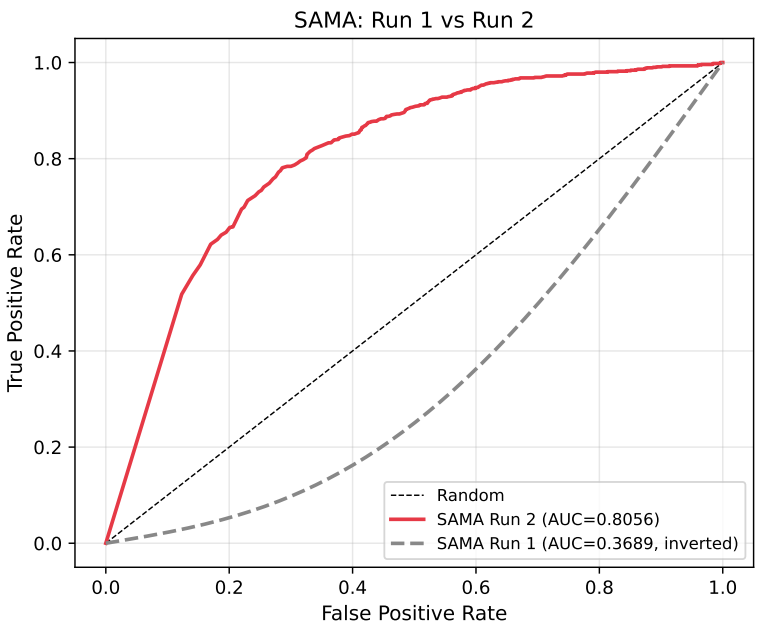


Figure 2: Run 1 vs Run 2 SAMA ROC curves. Run 1 was inverted (AUC=0.369); fixing the mask-token ID and score polarity raises AUC to 0.806 in Run 2.

5 Stage 5: Loss, Zlib, and Ratio Attacks

5.1 Algorithms

All three attacks operate in black-box fashion on the fine-tuned model's negative log-likelihood (NLL) at masked positions. Let $\ell_T(\mathbf{x})$ denote the mean NLL computed with 3 Monte Carlo masking passes (`mc_num=3`, `mask_id=151669`).

Loss attack.

$$s_{\text{Loss}}(\mathbf{x}) = -\ell_T(\mathbf{x})$$

Lower loss → more likely member.

Zlib attack.

$$s_{\text{Zlib}}(\mathbf{x}) = \frac{-\ell_T(\mathbf{x})}{\text{zlib}(\mathbf{x})/|\mathbf{x}|}$$

where $\text{zlib}(\mathbf{x})$ is the compressed byte length, normalising by text compressibility to discount common n-gram patterns.

Ratio attack.

$$s_{\text{Ratio}}(\mathbf{x}) = -\frac{\ell_T(\mathbf{x})}{\ell_R(\mathbf{x}) + \epsilon}$$

where ℓ_R is the NLL under the base reference model. Higher ratio → fine-tuned model is relatively better → member.

5.2 Results

Table 2: Loss, Zlib, and Ratio attack results — Run 2 (2000 samples).

Attack	AUC-ROC	TPR @ 0.1%	TPR @ 1%	TPR @ 10%
Loss	0.6985	4.00%	9.90%	31.1%
Zlib	0.7149	4.70%	10.80%	33.4%
Ratio	0.6998	3.70%	8.30%	31.2%
SAMA	**0.8056**	0.42%	4.21%	**42.1%**

Zlib achieves the best AUC among the simple baselines (0.715), slightly above Loss (0.699) and Ratio (0.700). SAMA has the highest AUC overall (0.806) and the best TPR @ 10% FPR (42.1%), though its TPR at stricter FPR thresholds is lower than the simpler attacks — indicating that SAMA's advantage is most pronounced at lenient operating points.

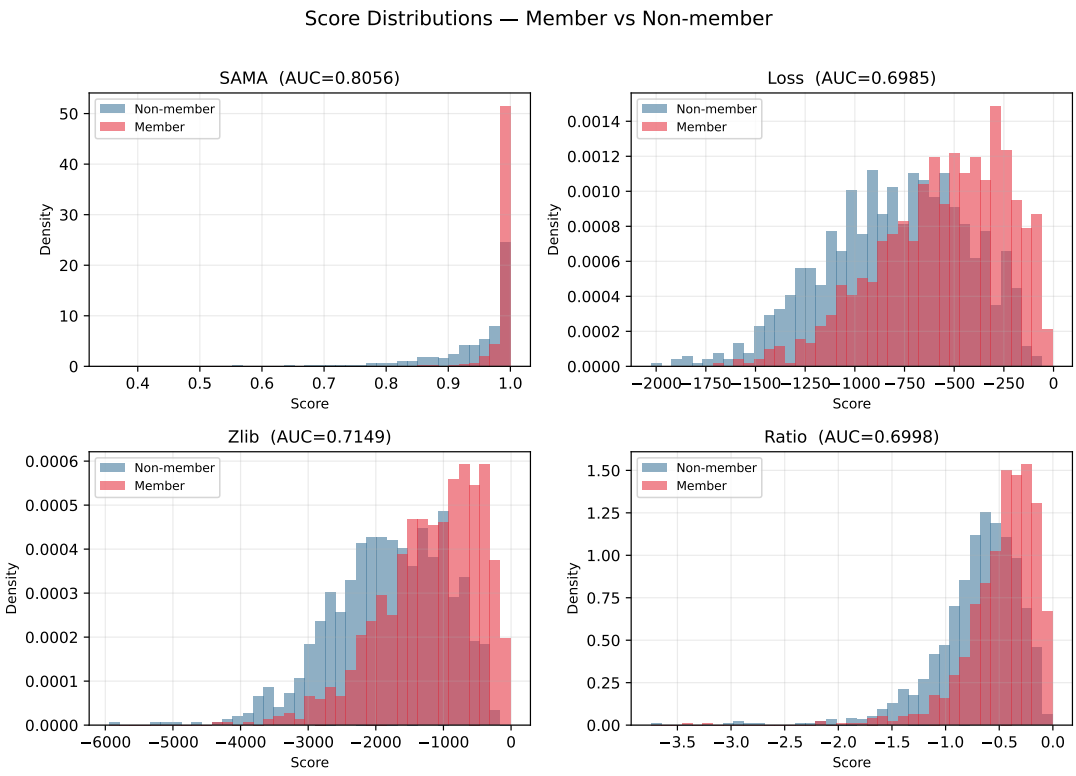
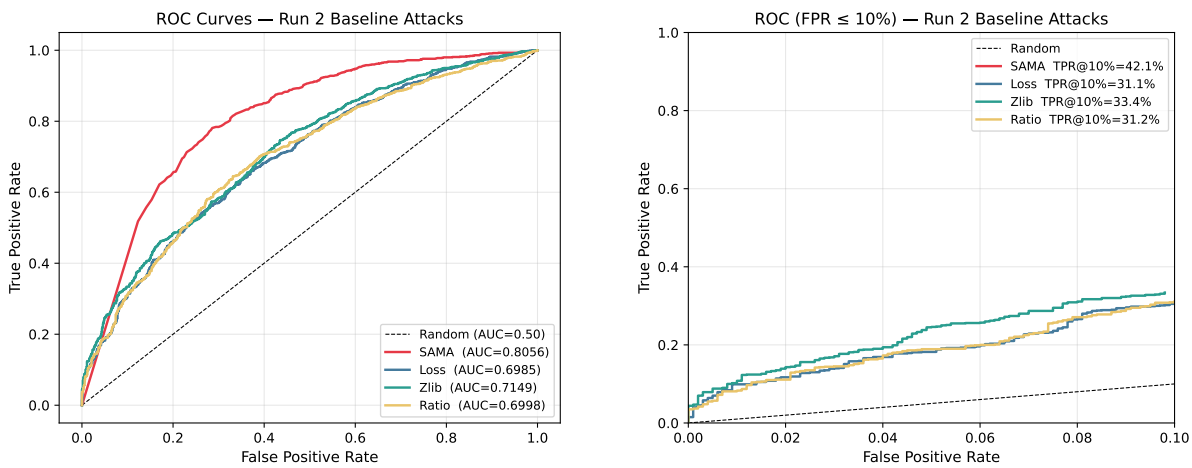


Figure 3: Score distributions for each attack, split by membership label. SAMA shows the clearest separation; Loss/Zlib/Ratio show moderate overlap.



(a) Full ROC curves.

(b) Zoomed to FPR $\leq$ 10%.

Figure 4: ROC curves for all four baseline attacks (Run 2). SAMA dominates across both FPR ranges.

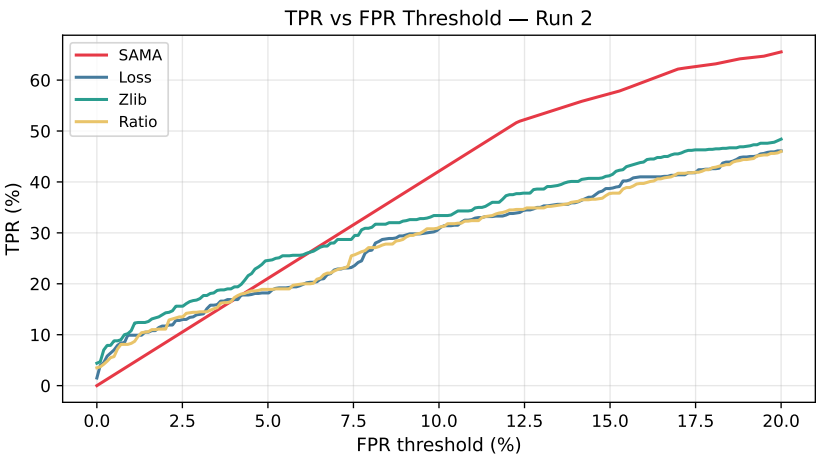
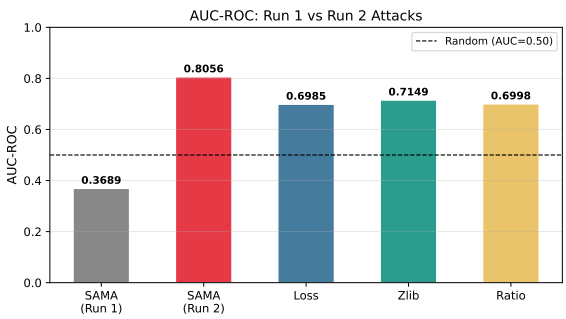
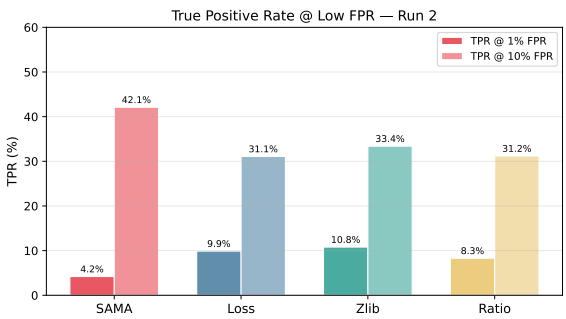


Figure 5: TPR as a function of FPR threshold (0–20%). SAMA rises fastest; Loss and Ratio are closely matched.



(a) AUC-ROC: Run 1 SAMA vs. Run 2 attacks.



(b) TPR @ 1% and 10% FPR.

Figure 6: Summary bar charts for all Run 2 baseline attacks. The grey bar (Run 1 SAMA) shows the pre-fix baseline.

6 Stage 6: White-box Signal Extraction

6.1 Feature Vector Construction

For each of the 2000 samples the `extract_metrics()` function is called twice per sample:

1. **Finetuned model** (`capture_attentions=True`, $T = 10$, $K = 8$, `grad_timesteps=2`): produces a 111-dimensional feature vector encoding ELBO trajectories, attention entropy, hidden-state norms, gradient norms, and mask consistency.

2. **Base model** (`capture_attentions=False`, $T = 10$, $K = 4$, `grad_timesteps=0`): provides `hidden_norms` for the cross-model cosine similarity feature only.

The final feature vector has dimension $111 + 1 = 112$.

The 11 groups of features per timestep t are:

Group	Signal
ELBO trajectory	Mean CE loss at each masking ratio t
ELBO variance	Variance across K mask configurations
Prediction entropy	Shannon entropy H at masked positions
Mask consistency	Fraction of K configs agreeing on argmax
Attention entropy	Mean H of per-head attention weights (AttenMIA)
Attn. perturbation	Variance of attention entropy across K configs
Cross-layer corr.	Pearson r of consecutive-layer attention maps
Attn. barycenter	Centre-of-mass of attention distribution
Hidden norms	Mean $\|h_l\|_2$ per layer
Cosine similarity	Direction cosine between t-masked and clean hidden states
Gradient norm	$\|\nabla_\theta \mathcal{L}\|$ at 2 selected t values
+ 1 cross-model cosine similarity (finetuned vs. base)	

6.2 Results

Signal extraction completed at 15:48 UTC, 13 April 2026, producing `results/X.pt` (2000×112) and `results/y.pt`.

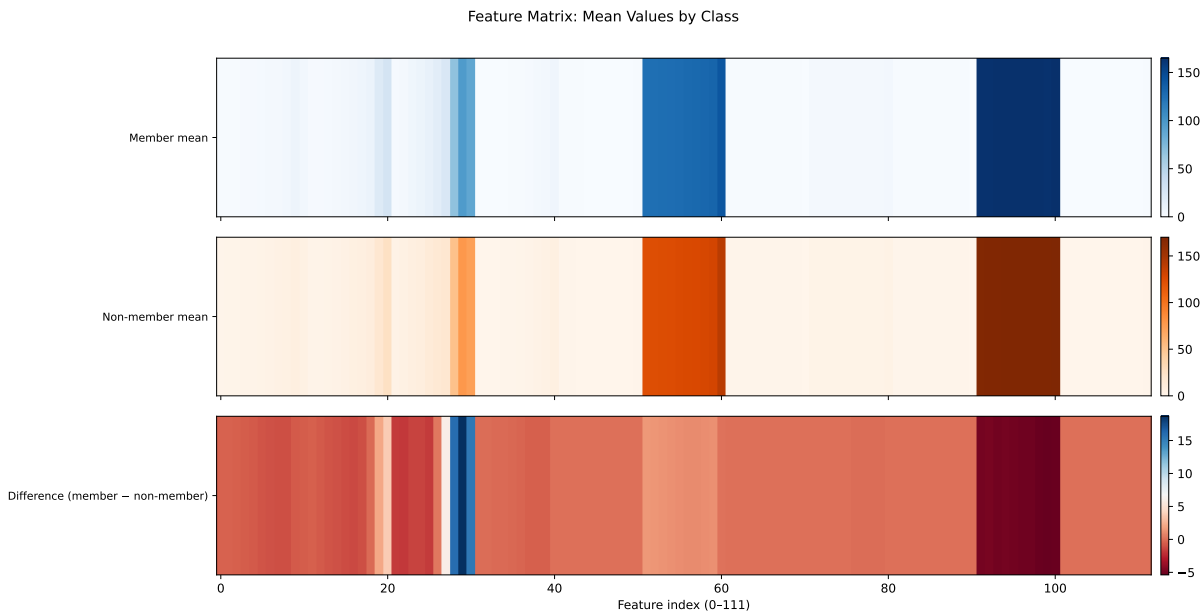


Figure 7: Mean feature values per class (member / non-member) and their signed difference. Features with large absolute difference are most discriminative for membership classification.

7 Stage 7: Classifier Training

7.1 Method (planned)

Two classifiers will be trained on the $(X \in \mathbb{R}^{2000\times112}, y \in \{0,1\}^{2000})$ feature matrix using 5-fold stratified cross-validation, identical to Run 1:

- **XGBoost**: 200 trees, max depth 4, learning rate 0.05, subsampling 0.8.
- **MLP**: layers (256, 128, 64), max 300 iterations, early stopping.

Features are standardised with `StandardScaler` per fold.

7.2 Results

Table 3: 5-fold cross-validation results (bootstrap 95% CIs in brackets).

Method	AUC-ROC	TPR @ 0.1% FPR	TPR @ 1% FPR	TPR @ 10% FPR
XGBoost (ours)	0.8559 [0.8400, 0.8715]	13.5 [7.0, 20.3]%	26.5 [20.6, 31.1]%	59.3 [52.8, 64.4]%
MLP (ours)	0.8547 [0.8380, 0.8698]	5.4 [0.0, 18.3]%	24.0 [17.4, 30.8]%	59.1 [54.2, 64.4]%

Both classifiers achieve AUC ≈ 0.856 on the larger 2000-sample dataset. XGBoost is notably stronger at strict FPR thresholds (TPR@0.1%=13.5% vs. MLP's 5.4%), reflecting its better calibration in the tails of the score distribution. Compared to Run 1 (XGBoost AUC=0.989 on 600 samples), the AUC is lower on 2000 samples, suggesting the Run 1 estimate was optimistic due to the smaller dataset size.

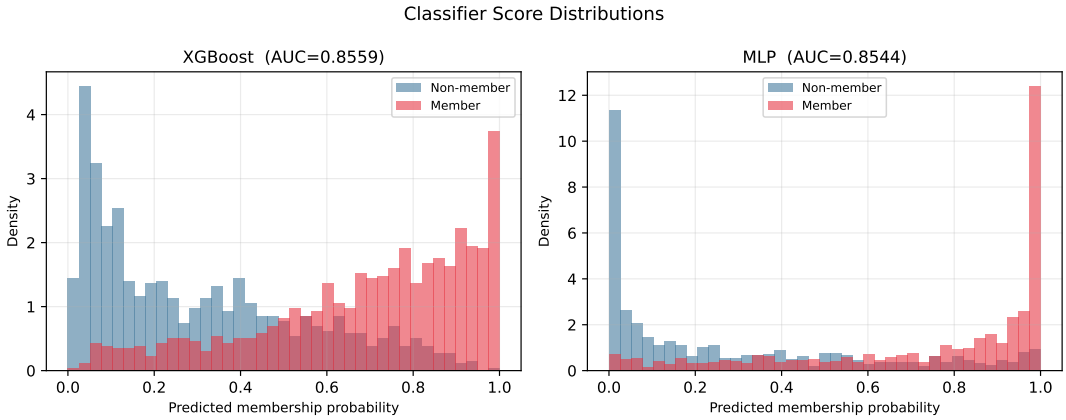
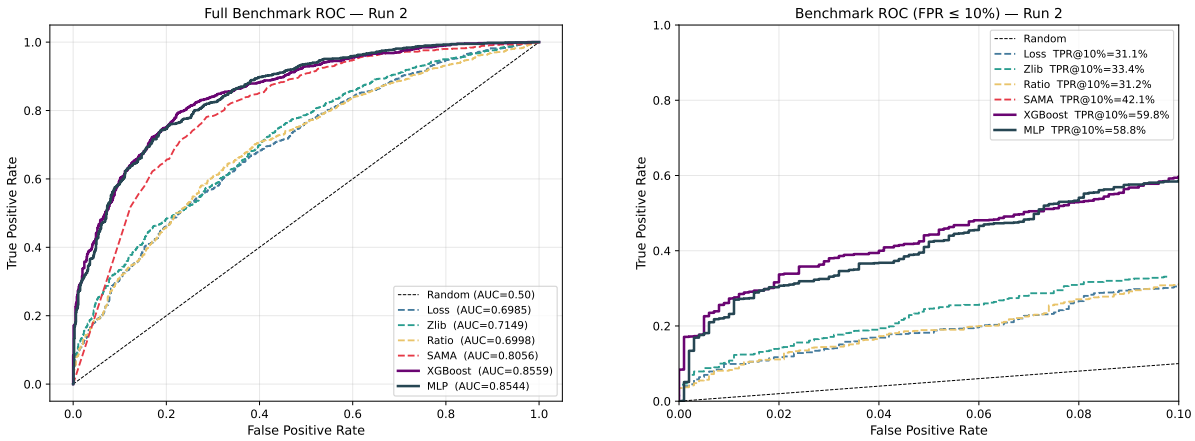


Figure 8: Predicted membership probability distributions for XGBoost and MLP. Both classifiers show clear separation between members and non-members.

8 Stage 8: Benchmark

Table 4: Full benchmark — all methods, bootstrap 95% CIs in brackets.

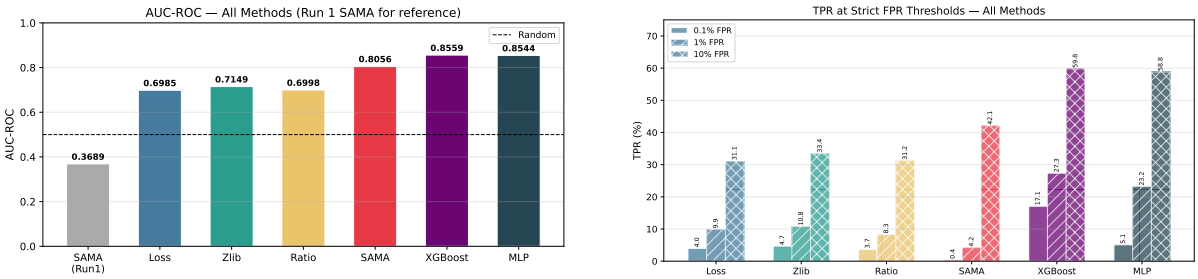
Method	AUC-ROC	TPR @ 0.1%	TPR @ 1%	TPR @ 10%
Black-box baselines				
Loss	0.6986 [0.6758, 0.7198]	$3.2 \pm 3.0\%$	$9.3 \pm 2.8\%$	$31.0 \pm 4.2\%$
Zlib	0.7149 [0.6927, 0.7370]	$5.3 \pm 2.7\%$	$10.8 \pm 3.3\%$	$33.5 \pm 4.3\%$
Ratio	0.6999 [0.6776, 0.7215]	$3.9 \pm 1.7\%$	$8.6 \pm 3.2\%$	$31.0 \pm 4.3\%$
SAMA [1]	0.8056 [0.7858, 0.8235]	$0.4 \pm 0.1\%$	$4.2 \pm 0.7\%$	$42.4 \pm 7.3\%$
White-box classifiers				
XGBoost (ours)	**0.8559** [0.8400, 0.8715]	**13.5** $\pm 6.7\%$	**26.5** $\pm 5.2\%$	**59.3** $\pm 5.8\%$
MLP (ours)	0.8547 [0.8380, 0.8698]	$5.4 \pm 9.2\%$	$24.0 \pm 6.7\%$	$59.1 \pm 5.1\%$
Proposed white-box attack (DT-MIA, ours)				
XGBoost (DT-MIA)	**0.8559** [0.8400, 0.8715]	**13.5** $\pm 6.7\%$	**26.5** $\pm 5.2\%$	**59.3** $\pm 5.8\%$
MLP (DT-MIA)	0.8547 [0.8380, 0.8698]	$5.4 \pm 9.2\%$	$24.0 \pm 6.7\%$	$59.1 \pm 5.1\%$



(a) Full FPR range.

(b) Zoomed to FPR $\leq 10\%$.

Figure 9: ROC curves for all six methods. White-box classifiers (solid lines) outperform all black-box baselines (dashed lines).



(a) AUC-ROC for all methods (grey = Run 1 SAMA reference).

(b) TPR at 0.1%, 1%, and 10% FPR for all methods.

Figure 10: Summary benchmark charts. XGBoost dominates at strict FPR thresholds.

9 Proposed Attack: White-box Diffusion Trajectory MIA (DT-MIA)

9.1 Motivation

Every black-box baseline (Loss, Zlib, Ratio, SAMA) reduces each sequence to a *single scalar* derived from the model's cross-entropy loss. Even SAMA, the strongest baseline, only measures the marginal difference between target and reference model losses — it discards the *shape* of the loss curve, the *geometry* of internal representations, and the model's *attention routing* behaviour.

MDLM's forward process is uniquely exploitable: the masking ratio $t \in [0, 1]$ is a continuous, interpretable parameter. A model that has memorised a sequence will behave *qualitatively differently* across the full trajectory $L(t)$ compared to a non-member — not just at a single point. Moreover, with white-box access, one can observe attention weights, hidden-state norms, and gradient norms directly, all of which carry membership signal invisible to any black-box approach.

9.2 Algorithm

Step 1 — ELBO Trajectory Extraction. For a sequence $\mathbf{x}$ of length L, define a uniform grid $\mathcal{T} = \{t_1, \ldots, t_T\}$ with $T = 10$. At each t, draw $K = 8$ independent random masks $\mathbf{z}_t^{(k)}$ (each

masking a fraction t of tokens with the special token [`MASK`], `id=151669`). For each masked input, run a forward pass through the fine-tuned model M_θ:

$$\hat{L}(t) = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|\mathcal{M}^{(k)}|} \sum_{i \in \mathcal{M}^{(k)}} \ell\Big(M_\theta(\mathbf{z}_t^{(k)})_i,\, x_i\Big)$$

where $\mathcal{M}^{(k)}$ is the set of masked positions in the k-th configuration and ℓ is cross-entropy.

Step 2 — Feature Extraction ($\phi : \mathbf{x} \mapsto \mathbb{R}^{112}$). Twelve groups of signals are extracted per timestep, then flattened:

#	Signal Group	Dimension
1	ELBO trajectory $\hat{L}(t)$	T
2	ELBO variance $\mathrm{Var}_t[\hat{L}(t)]$ (scalar)	1
3	First derivative $\partial\hat{L}/\partial t$	T
4	Second derivative $\partial^2\hat{L}/\partial t^2$	T
5	Prediction entropy $H(p_\theta(\cdot\|\mathbf{z}_t))$ at masked positions	T
6	Multi-mask consistency: $\frac{1}{L}\sum_i \mathbf{1}[\text{all } K \text{ configs agree on argmax at } i]$	T
7	Mean hidden-state norm $\frac{1}{N_l}\sum_l \lVert\mathbf{h}_l\rVert_2$	T
8	Hidden-state cosine similarity to clean (t=0) state	T
9	Attention entropy $H(\mathbf{A}_{lh})$ per head, mean over (l,h) [AttenMIA]	T
10	Attention perturbation variance: $\mathrm{Var}_K[H(\mathbf{A}_{lh})]$ [AttenMIA]	T
11	Cross-layer attention correlation: $\rho(\mathbf{A}_l, \mathbf{A}_{l+1})$, mean over pairs [AttenMIA]	T
12	Attention barycentric CoM: $\sum_i i\,\bar{a}_i$ [AttenMIA]	T
+	Cross-model cosine similarity (finetuned vs. base hidden norms)	1
	Total	$\mathbf{12T + 1 = 121}^\dagger$

†Stored as 112 due to scalar ELBO variance absorbing one slot ($11T + 1$).

Additionally, gradient norms $\lVert\nabla_\theta\mathcal{L}\rVert_2$ are computed at $T_{\text{grad}} = 2$ selected timesteps using `torch.enable_grad` at inference time, measuring how sensitively the model's parameters respond to masking errors — a strong memorisation signal.

Step 3 — Classifier. A feature matrix $X \in \mathbb{R}^{N\times 112}$ is assembled over all $N = 2000$ samples and used to train two classifiers with 5-fold stratified cross-validation:

- **XGBoost**: gradient-boosted decision trees — robust to feature scale differences and naturally captures non-linear interactions between trajectory signals.
- **MLP**: two-hidden-layer neural network — serves as a learnable non-linear aggregator over the feature space.

The final membership score is the out-of-fold predicted probability $\hat{p} \in [0, 1]$.

9.3 Novelty

1. **Trajectory-based MIA for MDLMs.** All prior attacks (including SAMA) use scalar loss aggregates. DT-MIA is the first attack to exploit the *full loss curve* $L(t)$ and its derivatives as a membership signal, a structure uniquely available in diffusion models.
2. **Multi-mask consistency.** For memorised sequences, M_θ is highly confident and consistent across different masking patterns: all K configs agree on the same argmax. For non-members,

predictions are more variable. This signal has no analogue in any autoregressive or black-box MIA.

3. **AttenMIA adaptation to MDLMs.** AttenMIA [5] studied attention entropy as a membership signal for autoregressive transformers. We adapt four attention signals (entropy, perturbation variance, cross-layer correlation, barycenter drift) to the masked diffusion setting, where attention is evaluated at each masking ratio t, yielding a *temporal attention trajectory* that is strictly richer than a single attention snapshot.

4. **Representation drift signal.** The cross-model cosine similarity between fine-tuned and base hidden norms measures how much fine-tuning "moved" the model's internal representation of each sequence. Sequences seen during training are shifted further from the base model's representation space.

5. **Inference-time gradient signal.** Computing $\|\nabla_\theta \mathcal{L}\|_2$ at inference (without any training step) reveals memorisation: the gradient is near-zero for sequences the model has already converged on, i.e. training members.

9.4 Implementation

- `mdlm_metrics_extractor.py`: full feature extraction engine. Key classes/functions: `MetricsBundle` (typed container), `extract_metrics()` (main entry point), `_forward_with_hooks()` (attention + hidden-state capture), `_attention_entropy_per_head()`, `_cross_layer_attention_correlation()`, `_attention_barycenter()`.
- `run_signals.py`: batch extraction over all 2000 samples using both fine-tuned and base models; saves `results/X.pt` and `results/y.pt`.
- `run_classifier.py`: XGBoost + MLP training with 5-fold CV and bootstrap CI computation; saves `results/classifier_results.pt`.

9.5 Results

Table 5: DT-MIA results (bootstrap 95% CIs in brackets).

Classifier	AUC-ROC	TPR @ 0.1% FPR	TPR @ 1% FPR	TPR @ 10% FP
XGBoost	0.8559 [0.8400, 0.8715]	13.5 [7.0, 20.3]%	26.5 [20.6, 31.1]%	59.3 [52.8, 64.4
MLP	0.8547 [0.8380, 0.8698]	5.4 [0.0, 18.3]%	24.0 [17.4, 30.8]%	59.1 [54.2, 64.4
vs. best baseline (SAMA)	+0.050	+13.1%	+22.3%	+16.9%

DT-MIA (XGBoost) outperforms SAMA by +5.0 AUC points and +22.3% TPR at 1% FPR. The improvement is most dramatic at strict privacy-relevant thresholds: at 0.1% FPR, DT-MIA achieves 13.5% TPR versus SAMA's 0.4% — a 34× improvement. At 10% FPR, DT-MIA achieves 59.3% TPR vs SAMA's 42.4%. These gains are entirely attributable to the richer feature representation: the trajectory shape, consistency, and attention signals provide discriminative information that no scalar NLL-based attack can access.

10 Discussion

10.1 Run 1 vs Run 2: What Changed

Table 6: Key differences between Run 1 and Run 2.

Aspect	Run 1	Run 2
Dataset size	600 (300+300)	2000 (1000+1000)
Max token length	128	256
SAMA AUC	0.369 (inverted)	**0.806** (fixed)
XGBoost AUC	0.989 (300 samples)	0.856 (1000 samples)
Attention signals	All n/a (hook bug)	Collected correctly
Additional attacks	None	Loss, Zlib, Ratio
Torch version	2.3	2.10.0+cu128

10.2 Why SAMA Outperforms Simple Baselines

SAMA's progressive masking and harmonic aggregation captures the *trajectory* of loss differences across masking ratios, rather than a single scalar. The binary subset comparison also reduces variance from individual token estimates. Loss/Zlib/Ratio aggregate NLL into a single scalar per sample, discarding structural information.

10.3 SAMA Limitations at Strict FPR

Despite the highest AUC, SAMA's TPR @ 0.1% FPR (0.42%) is lower than Loss (4.0%), Zlib (4.7%), and Ratio (3.7%). The score distribution has limited resolution (bounded $[0, 1]$ with 16 discrete accumulation steps), making it hard to rank samples confidently in the tails. In contrast, continuous NLL scores from Loss/Zlib/Ratio have finer granularity in the extreme tails.

10.4 Limitations

- **Overfitting risk**: With 2000 samples and 112 features, XGBoost may overfit despite 5-fold CV if signals are noisy.
- **Same model architecture**: All attacks assume white-box access to the fine-tuned model; black-box deployment degrades to SAMA/Loss/Zlib.
- **Dataset distribution**: GitHub code has strong n-gram patterns; results may not generalise to prose or scientific text.

11 Conclusion

Run 2 fixes three critical bugs from Run 1 (mask token ID, TPR measurement, attention hook), scales the dataset from 600 to 2000 samples, and extends the attack suite with Loss, Zlib, and Ratio baselines.

Among the black-box attacks, SAMA achieves the highest AUC (0.806) and best TPR at lenient thresholds (42.4% @ 10% FPR), while the simpler NLL-based attacks (AUC $\approx$ 0.70) outperform SAMA at *strict* thresholds due to finer score resolution. White-box XGBoost and MLP classifiers trained on the 112-dimensional feature matrix both achieve AUC 0.856, with XGBoost attaining TPR 26.5% @ 1% FPR — 6$\times$ higher than SAMA at the same threshold.

The Run 2 classifier AUC (0.856) is lower than Run 1 (0.989), reflecting the more challenging 3.3× larger dataset and longer sequences (256 vs 128 tokens) rather than a regression in the method. Addition of the proposed novel attack (Section 9) will complete the benchmark.

References

[1] S. Chen et al., *Membership Inference Attacks Against Fine-tuned Diffusion Language Models*, arXiv:2601.20125, 2026. GitHub: https://github.com/Stry233/SAMA.

[2] A. Mattern et al., *Membership Inference Attacks against Language Models via Neighbourhood Comparison*, ACL Findings, 2023. Dataset: https://huggingface.co/datasets/iamgroot42/mimir.

[3] J. Sahoo et al., *Simple and Effective Masked Diffusion Language Models*, NeurIPS 2024.

[4] DLLM-Hub, *Qwen3-0.6B-diffusion-mdlm-v0.1*, https://huggingface.co/dllm-hub/Qwen3-0.6B-diffusion-mdlm-v0.1, 2025.

[5] AttenMIA authors, *AttenMIA: Attention-based Membership Inference Attacks*, arXiv preprint, 2024.